

ASSIGNMENT-6.3

Name:Sushmitha Uppula

Batch:24

Ht.No:2303A51705

Task Description #1 (Loops – Automorphic Numbers in a Range)

- Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

- Instructions:

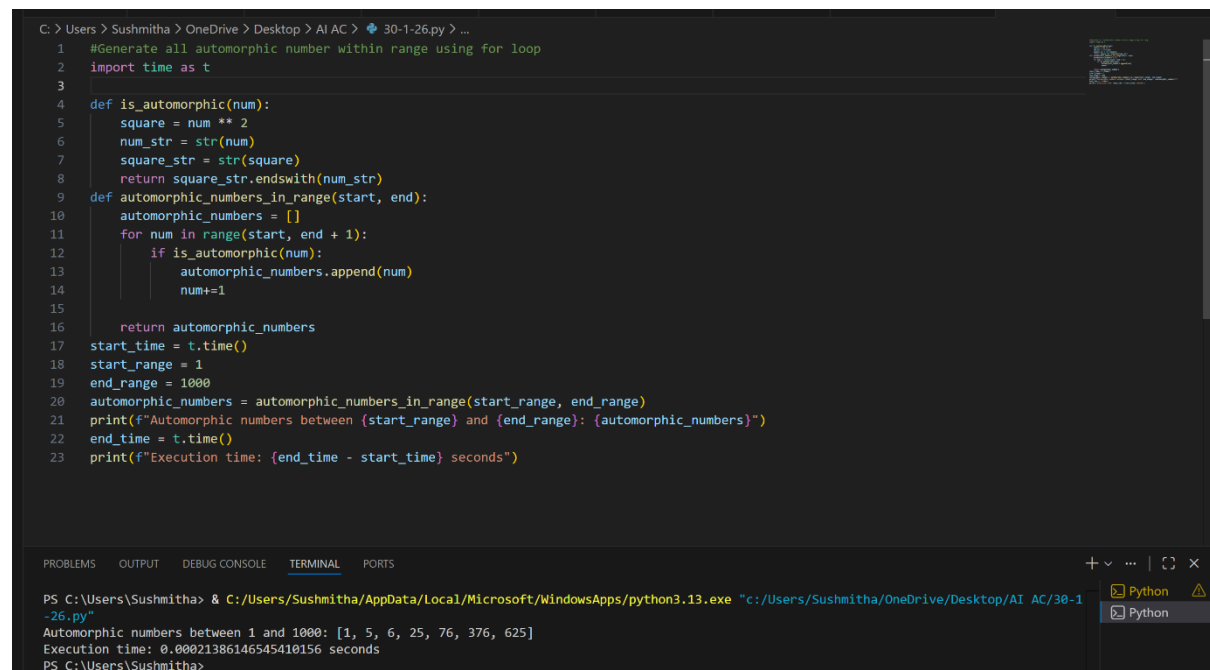
- o Get AI-generated code to list Automorphic numbers using a for loop.

- o Analyze the correctness and efficiency of the generated logic.

- o Ask AI to regenerate using a while loop and compare both implementations.

Expected Output #1:

- Correct implementation that lists Automorphic numbers using both loop types, with explanation.



```
C:\> Users\ Sushmitha > OneDrive > Desktop > AI AC > 30-1-26.py > ...
1  #Generate all automorphic number within range using for loop
2  import time as t
3
4  def is_automorphic(num):
5      square = num ** 2
6      num_str = str(num)
7      square_str = str(square)
8      return square_str.endswith(num_str)
9
10 def automorphic_numbers_in_range(start, end):
11     automorphic_numbers = []
12     for num in range(start, end + 1):
13         if is_automorphic(num):
14             automorphic_numbers.append(num)
15             num+=1
16
17     return automorphic_numbers
18
19 start_time = t.time()
20 start_range = 1
21 end_range = 1000
22 automorphic_numbers = automorphic_numbers_in_range(start_range, end_range)
23 print(f"Automorphic numbers between {start_range} and {end_range}: {automorphic_numbers}")
24 end_time = t.time()
25 print(f"Execution time: {end_time - start_time} seconds")
```

```
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/30-1-26.py"
Automorphic numbers between 1 and 1000: [1, 5, 6, 25, 76, 376, 625]
Execution time: 0.00021386146545410156 seconds
PS C:\Users\Sushmitha>
```

Classification)

- Instructions:

- Expected Output #2:

- ```
C:\Users\Sushmitha> OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 2.py > ...
```
- ```
1 def shopping_feedback(feedback):
2     positive_words = {"good", "great", "excellent", "amazing", "fantastic", "satisfied", "happy", "love"}
3     negative_words = {"bad", "terrible", "poor", "disappointed", "hate", "awful", "worst", "unsatisfied"}
4     neutral_words = {"okay", "average", "fine", "decent", "mediocre"}
5
6     feedback_words = set(feedback.lower().split())
7
8     if feedback_words & positive_words:
9         return "Positive"
10    elif feedback_words & negative_words:
11        return "Negative"
12    else:
13        return "Neutral"
14    user_feedback = input("Enter your shopping feedback: ")
15    print("The feedback is:", shopping_feedback(user_feedback))
16
```
- PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
- PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/ASSIGNMENT_6.3/2.py"
- Enter your shopping feedback: the product was amazing
The feedback is: Positive
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/ASSIGNMENT_6.3/2.py"
- Enter your shopping feedback: the product was average
The feedback is: Neutral
PS C:\Users\Sushmitha>

Define a function named `statistical_operations(tuple_num)` that performs the following statistical operations on a tuple of numbers:

- Minimum, Maximum

- Mean, Median, Mode
- Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub

Copilot. Make decisions to accept, reject, or modify the suggestions based on

their relevance and correctness

```
C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 3.py > ...
1  def statistical_operations(tuple_num):
2      import statistics
3
4      mean = statistics.mean(tuple_num)
5      median = statistics.median(tuple_num)
6      try:
7          mode = statistics.mode(tuple_num)
8      except statistics.StatisticsError:
9          mode = "No unique mode found"
10     maximum = max(tuple_num)
11     minimum = min(tuple_num)
12     variance = statistics.variance(tuple_num)
13     std_dev = statistics.stdev(tuple_num)
14
15     return mean, median, mode, maximum, minimum, variance, std_dev
16 tuple_num = (1, 2, 2, 3, 4, 5, 5, 5)
17 mean, median, mode, maximum, minimum, variance, std_dev = statistical_operations(tuple_num)
18 print(f"Mean: {mean}")
19 print(f"Median: {median}")
20 print(f"Mode: {mode}")
21 print(f"Maximum: {maximum}")
22 print(f"Minimum: {minimum}")
23 print(f"Variance: {variance}")
24 print(f"Standard Deviation: {std_dev}")
25
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/
Mean: 3.375
Median: 3.5
Mode: 5
Maximum: 5
Minimum: 1
Variance: 2.5535714285714284
Standard Deviation: 1.5979898086569353
PS C:\Users\Sushmitha>
```

Task 4: Teacher Profile

- Prompt: Create a class Teacher with attributes teacher_id, name, subject, and experience. Add a method to display teacher details.
- Expected Output: Class with initializer, method, and object creation.

```
C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 4.py > ...
1  #Create a class Teacher with attributes teacher_id, name,subject, and experience. Add a method to display teacher details.
2  class Teacher:
3      def __init__(self, teacher_id, name, subject, experience):
4          self.teacher_id = teacher_id
5          self.name = name
6          self.subject = subject
7          self.experience = experience
8
9      def display_details(self):
10         print(f"Teacher ID: {self.teacher_id}")
11         print(f"Name: {self.name}")
12         print(f"Subject: {self.subject}")
13         print(f"Experience: {self.experience} years")
14 # Example usage:
15 teacher = Teacher(1, "Alice Smith", "Mathematics", 10)
16 teacher.display_details()
17
18
19
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  Pyth

```
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/
Teacher ID: 1
Name: Alice Smith
Subject: Mathematics
Experience: 10 years
PS C:\Users\Sushmitha>
```

Task #5 – Zero-Shot Prompting with Conditional Validation

Use zero-shot prompting to instruct an AI tool to generate a function that validates an Indian mobile number.

Requirements

- The function must ensure the mobile number:
 - o Starts with 6, 7, 8, or 9
 - o Contains exactly 10 digits

Expected Output

- A valid Python function that performs all required validations without using any input-output examples in the prompt.

```
C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 5.py > ...
1 #generate a python program that validates an Indian mobile number that ensures the number starts with 6,7,8 0r 9 and contains exactly
2 import re
3
4 def validate_mobile_number(mobile_number):
5     pattern = r'^[6-9]\d{9}$'
6     if re.match(pattern, mobile_number):
7         return True
8     else:
9         return False
10
11 # Example usage:
12 mobile_number = input("enter mobile number: ")
13 if validate_mobile_number(mobile_number):
14     print("Valid Indian mobile number")
15 else:
16     print("Invalid Indian mobile number")
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + -

```
PS C:\Users\Sushmitha> C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/ASSIG
enter mobile number: sdfyguhl
Invalid Indian mobile number
PS C:\Users\Sushmitha> C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/OneDrive/Desktop/AI AC/ASSIG
enter mobile number: 7993555427
Valid Indian mobile number
PS C:\Users\Sushmitha>
```

Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000).

Instructions:

- Use a for loop and digit power logic.
- Validate correctness by checking known Armstrong numbers (153, 370, etc.).
- Ask AI to regenerate an optimized version (using list comprehensions).

Expected Output #7:

- Python program listing Armstrong numbers in the range.
- Optimized version with explanation.

C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 6.py > ...

```
1 def armstrong_number(num):
2     # Convert number to string to easily iterate over digits
3     str_num = str(num)
4     num_digits = len(str_num)
5     for digit in str_num:
6         if not digit.isdigit():
7             return "invalid input"
8
9     sum_of_powers = sum(int(digit) ** num_digits for digit in str_num)
10    return sum_of_powers == num
11    return False
12 if __name__ == "__main__":
13     user_input = input("Enter a number: ")
14     if not user_input.isdigit():
15         print("invalid input")
16     else:
17         number = int(user_input)
18         if armstrong_number(number):
19             print(f"{number} is an Armstrong number.")
20         else:
21             print(f"{number} is not an Armstrong number.")
22
23
24
25
26
27
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python

Enter a number: sfdh

invalid input

PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python

Enter a number: 153

153 is an Armstrong number.

PS C:\Users\Sushmitha>

> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python

Enter a number: 567

567 is not an Armstrong number.

PS C:\Users\Sushmitha> █

Task Description #7 (Loops – Happy Numbers in a Range)

Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

- Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy

Number) or enters a cycle (Not Happy).

- Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28...).
- Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).

Expected Output #8:

- Python program that prints all Happy Numbers within a range.
- Optimized version using cycle detection with explanation.

```
C:\Users\Sushmitha> OneDrive\Desktop> AI AC> ASSIGNMENT_6.3> 7.py > ...
1
2 def happy_number(n):
3     seen = set()
4     while n != 1 and n not in seen:
5         seen.add(n)
6         n = sum(int(digit) ** 2 for digit in str(n))
7     return n == 1
8 def happy_numbers_in_range(start, end):
9     happy_numbers = []
10    for num in range(start, end + 1):
11        if happy_number(num):
12            happy_numbers.append(num)
13    return happy_numbers
14 start_range = int(input("Enter the start of the range: "))
15 end_range = int(input("Enter the end of the range: "))
16 happy_nums = happy_numbers_in_range(start_range, end_range)
17 print(f"Happy Numbers between {start_range} and {end_range}: {happy_nums}")
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/7.py"
Enter the start of the range: 1
Enter the end of the range: 100
Happy Numbers between 1 and 100: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100]
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sushmitha/7.py"
Enter the start of the range: 50
Enter the end of the range: 20
Happy Numbers between 50 and 20: []
PS C:\Users\Sushmitha>
```

Task Description #8 (Loops – Strong Numbers in a Range)

Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., $145 = 1! + 4! + 5!$) within a given range.

Instructions:

- Use loops to extract digits and calculate factorials.
- Validate with examples (1, 2, 145).
- Ask AI to regenerate an optimized version (precompute digit factorials).

Expected Output #9:

- Python program that lists Strong Numbers.
- Optimized version with explanation.

C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 8.py > ...

```

1  import math
2  def strong_number(num):
3      sum_of_factorials = 0
4      temp = num
5      while temp > 0:
6          digit = temp % 10
7          sum_of_factorials += math.factorial(digit)
8          temp //= 10
9      return sum_of_factorials == num
10 num=int(input("Enter a number: "))
11 if strong_number(num):
12     print(f"{num} is a Strong number.")
13 else:
14     print(f"{num} is not a Strong number.")
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Enter a number: 1
1 is a Strong number.
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/Windows
Enter a number: 2
2 is a Strong number.
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/Windows
Enter a number: 145
145 is a Strong number.
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/Windows

Enter a number: 67
67 is not a Strong number.
PS C:\Users\Sushmitha>

```


Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

- The function should extract and return:

- o Full Name

- o Branch

- o SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples

```
C: > Users > Sushmitha > OneDrive > Desktop > AI AC > ASSIGNMENT_6.3 > 9.py > ...
1  #generate a python code that parses a nested dictionary representing student information.
2  #the dictionary contains Full Name Branch SGPA
3  def parse_student_info(student_dict):
4      for student_id, info in student_dict.items():
5          full_name = info.get("Full Name", "N/A")
6          branch = info.get("Branch", "N/A")
7          sgpa = info.get("SGPA", "N/A")
8          print(f"Student ID: {student_id}")
9          print(f"Full Name: {full_name}")
10         print(f"Branch: {branch}")
11         print(f"SGPA: {sgpa}")
12         print("-" * 20)
13 # Example usage
14 students = {
15     101: {"Full Name": "John Doe", "Branch": "Computer Science", "SGPA": 8.5},
16     102: {"Full Name": "Jane Smith", "Branch": "Electrical Engineering", "SGPA": 9.0},
17     103: {"Full Name": "Alice Johnson", "Branch": "Mechanical Engineering", "SGPA": 8.8},
18 }
19 parse_student_info(students)
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/

Student ID: 101
Full Name: John Doe
Branch: Computer Science
SGPA: 8.5
-----
Student ID: 102
Full Name: Jane Smith
Branch: Electrical Engineering
SGPA: 9.0
-----
Student ID: 103
Full Name: Alice Johnson
Branch: Mechanical Engineering
SGPA: 8.8
-----
PS C:\Users\Sushmitha>
```

Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

- A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).
- o Example: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$.
- Use a for loop to find divisors of each number in the range.
- Validate correctness with known Perfect Numbers (6, 28, 496...).
- Ask AI to regenerate an optimized version (using divisor check only up to $\sqrt{n}$).

Expected Output #12:

- Python program that lists Perfect Numbers in the given range.
- Optimized version with explanation.

```
C:\> Users > Sushmitha > OneDrive > Desktop > AI AC > Assignment_4 > 10.py > ...
1  def perfect_numbers(n):
2      perfect_nums = []
3      for num in range(1, n + 1):
4          sum_of_divisors = 0
5          for i in range(1, num):
6              if num % i == 0:
7                  sum_of_divisors += i
8              if sum_of_divisors == num:
9                  perfect_nums.append(num)
10     return perfect_nums
11 # Example usage:
12 n = int(input("Enter a number: "))
13 print(f"Perfect numbers up to {n}: {perfect_numbers(n)}")
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter a number: 6
Perfect numbers up to 6: [6]
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3
Enter a number: 28
Perfect numbers up to 28: [6, 28]
PS C:\Users\Sushmitha> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3
Enter a number: 50
Perfect numbers up to 50: [6, 28]
PS C:\Users\Sushmitha>
```